

The Transformer Architecture

Chapter 7 · Building the complete block

Dr. A. Belcaid

Table of contents

1 Position: attention needs an order **3**
1.1 The permutation problem . . . 3
1.2 Add content and position . . . 3
1.3 Sinusoidal positions . . . 4

2 Transform features: the position-wise FFN **4**

3 Decode without looking ahead **5**

4 Build depth safely **6**
4.1 Multi-head attention revisited . . . 6
4.2 Residual connections . . . 6
4.3 Layer normalization . . . 6

5 Encoder and decoder stacks **8**
5.1 The decoder . . . 8
5.2 Why the encoder has no causal mask . . . 9

6 Cross-attention connects both towers **10**

7 Three Transformer families **12**

8 Evidence and limitations **13**
8.1 Translation results . . . 13
8.2 Transfer across tasks . . . 13
8.3 Full attention is quadratic . . . 13

9 Synthesis **14**
9.1 Further reading . . . 14

References **14**

Attention gives a model a powerful communication rule: each token can retrieve a weighted mixture of other representations. A complete Transformer needs more. It must encode order, transform each retrieved message nonlinearly, prevent information leakage during generation, and remain trainable when many blocks are stacked.

This chapter follows the lecture's construction rather than presenting the final diagram all at once. Each addition repairs one concrete limitation. By the end, the encoder, decoder, and encoder–decoder Transformer will be consequences of a few explicit design choices.

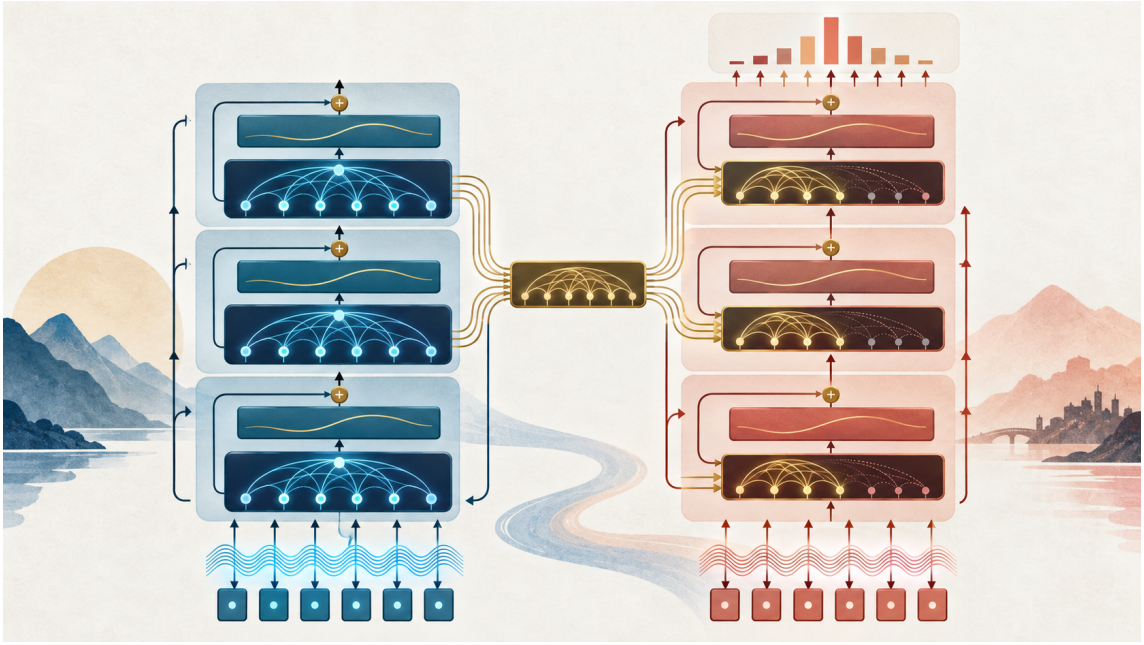


Figure 1: The complete conceptual journey: position-aware inputs enter encoder and decoder stacks, encoder memory crosses into the decoder, and the decoder produces an output distribution.

Learning objectives

After this chapter, you should be able to:

1. explain why self-attention needs positional information;
2. compute sinusoidal positional encodings;
3. describe the role of the position-wise feed-forward network;
4. construct a causal attention mask;
5. explain residual connections and LayerNorm;
6. distinguish encoder, decoder, and encoder–decoder blocks;
7. identify the origins of queries, keys, and values in cross-attention;
8. select an architecture family for a task; and
9. derive the quadratic cost of full self-attention.

1 Position: attention needs an order

1.1 The permutation problem

Compare “Alice thanked Bob” with “Bob thanked Alice.” The token identities are the same, but their roles are reversed. Without a separate position signal, self-attention is **permutation equivariant**. If a permutation matrix P reorders the rows of X, then

$$\mathrm{Attn}(PX) = P\,\mathrm{Attn}(X).$$

The outputs follow the reordered inputs, but the attention mechanism has no independent fact saying which occurrence came first. This is why the original Transformer adds position representations before the first layer (Vaswani et al. 2017). See also Stanford CS224N Lecture 5 and The Illustrated Transformer.

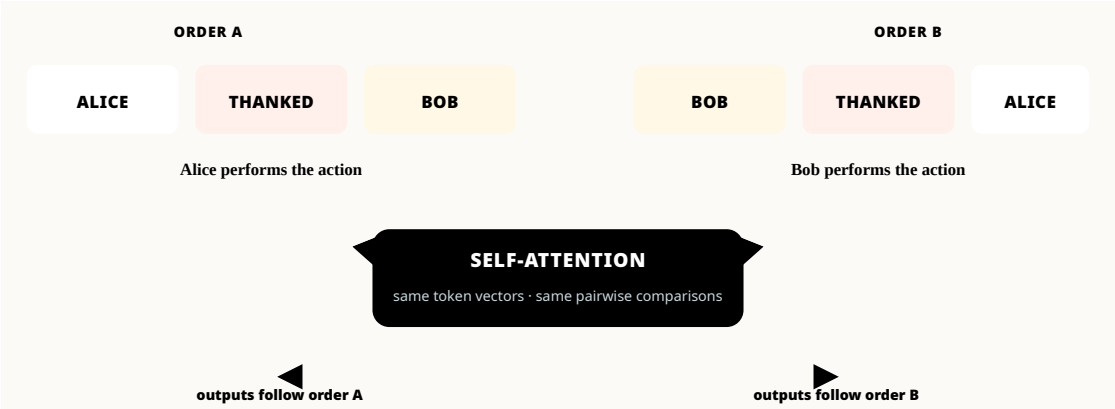


Figure 2: Without a position signal, self-attention follows a permutation but cannot identify the changed order. This standalone vector figure is generated from the lecture’s original SVG code.

1.2 Add content and position

For token i, the Transformer input is

$$\boxed{z_i = e(\mathrm{token}_i) + p_i}. \tag{1}$$

Both vectors have dimension d_{model}, so addition preserves one vector per token. The word *cats* keeps the same lexical embedding wherever it occurs, but adding p_1 or p_3 produces different Transformer inputs.

> **!** The positional vector does not replace the token embedding. It perturbs it with an order-dependent signal while preserving the interface expected by attention.

1.3 Sinusoidal positions

The classical fixed encoding uses paired sine and cosine waves:

$$\mathrm{PE}(pos, 2i) = \sin\left(pos / 10000^{2i/d_{\text{model}}}\right),$$

$$\mathrm{PE}(pos, 2i+1) = \cos\left(pos / 10000^{2i/d_{\text{model}}}\right).$$

Each feature uses a different wavelength. Nearby positions change smoothly, while the collection of frequencies gives each position a distinctive pattern. The design also makes fixed offsets expressible through linear combinations of sine and cosine terms.

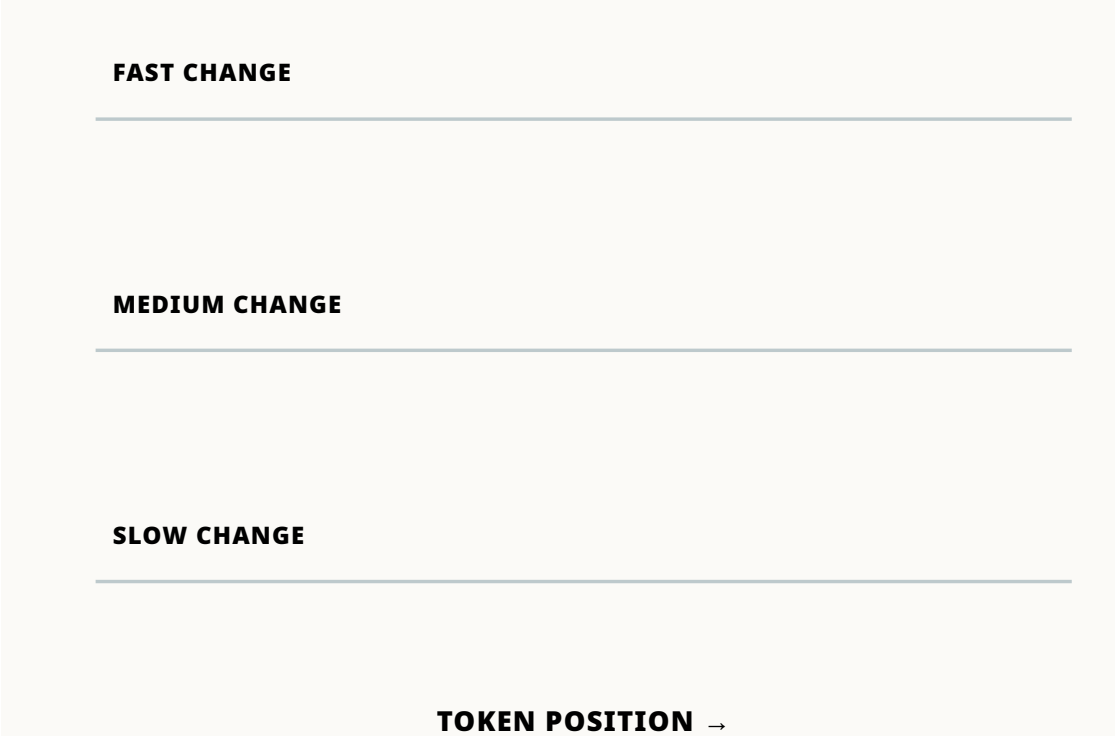


Figure 3: Sinusoidal position encoding: waves at different frequencies, rendered directly from the lecture's SVG source.

Other approaches include learned absolute embeddings, relative position representations, Transformer-XL relative attention, RoPE, and ALiBi. We focus here on the original sinusoidal construction because it makes the architectural role of position especially clear.

2 Transform features: the position-wise FFN

Attention communicates across tokens through weighted sums. If we only stack linear weighted mixing operations, the model lacks a strong nonlinear transformation at each position. The Transformer therefore applies the same two-layer network independently to every token:

$$\mathrm{FFN}(x) = W_2\,\sigma(W_1 x + b_1) + b_2. \tag{2}$$

In the original architecture, $sigma$ is ReLU, W_1 expands from d_{model} to d_{ff}, and W_2 projects back. Modern variants often use GELU, SwiGLU, or gated variants.

Sublayer	Operates across	Main role
Self-attention	token positions	communicate and retrieve
Position-wise FFN	feature dimensions	nonlinear transformation

The same FFN parameters are reused at every position **within a layer**. Different Transformer layers have different learned parameters.

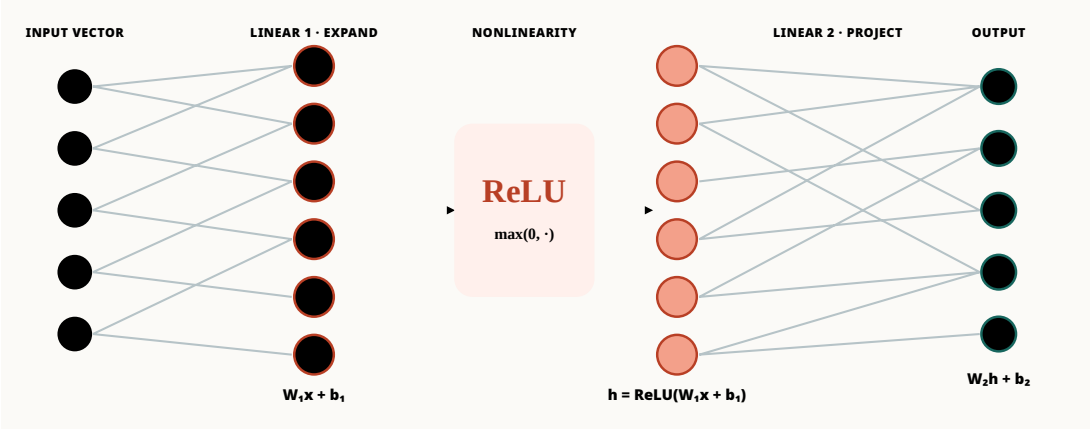


Figure 4: The lecture's neural-network view of the position-wise FFN: a first affine layer, ReLU, and a second affine layer return each token to the model dimension.

3 Decode without looking ahead

During autoregressive generation, predicting token y_t may depend on $y_{<t}$ but not on future answers $y_{>t}$. The decoder implements this rule before softmax:

$$M_{ij} = \begin{cases} 0, & j \le i, \\ -\infty, & j > i, \end{cases} \qquad A = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right). \tag{3}$$

Because $e^{-\infty} = 0$, forbidden positions receive zero probability. Applying the mask **after** softmax is incorrect: the remaining weights would no longer be normalized.

```
scores = q @ k.transpose(-2, -1) / math.sqrt(d_k)
mask = torch.triu(torch.ones(T, T, dtype=torch.bool), diagonal=1)
scores = scores.masked_fill(mask, float("-inf"))
weights = torch.softmax(scores, dim=-1)
output = weights @ v
```

Compare this explicit calculation with `torch.nn.functional.scaled_dot_product_attention`, which supports causal attention directly.

4 Build depth safely

4.1 Multi-head attention revisited

One head produces one attention distribution per query. Different heads have separate projection matrices and may learn different relations—for example, a subject–verb relation in one head and an entity-reference relation in another:

$$\text{MHA}(X) = \text{Concat}(head_1, \dots, head_h)W_O.$$

This is a capacity argument, not a guarantee that every head receives a clean linguistic interpretation. Clark et al. analyze BERT attention patterns; BertViz provides an interactive inspection tool.

4.2 Residual connections

A residual sublayer learns a correction to its input:

$$y = x + F(x).$$

The identity path carries representations and gradients through depth even when the learned branch is initially unhelpful. The loss-landscape visual used in the lecture comes from Li et al., *Visualizing the Loss Landscape of Neural Nets*.

4.3 Layer normalization

For one token vector $x \in \mathbb{R}^d$, LayerNorm computes statistics across its **feature dimension**:

$$\mu = \frac{1}{d}\sum_{j=1}^{d} x_j, \qquad \sigma^2 = \frac{1}{d}\sum_{j=1}^{d}(x_j - \mu)^2,$$

$$\text{LN}(x)_j = \gamma_j \frac{x_j - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta_j. \tag{4}$$

γ and β are learned feature-wise parameters. Statistics are computed independently for each token and example—not across tokens in the batch. Consult the original Layer Normalization paper and the `torch.nn.LayerNorm` documentation.

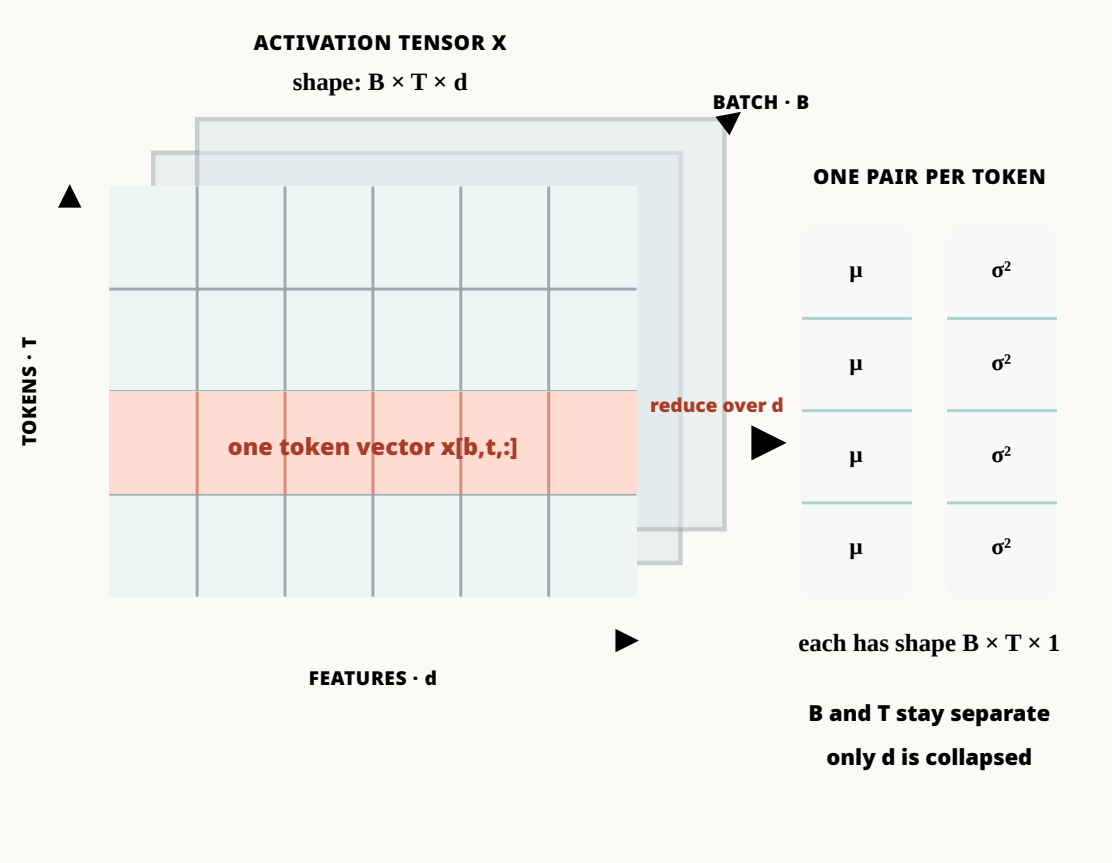


Figure 5: LayerNorm operates along the feature dimension of each token representation; it does not aggregate across tokens or batches.

```python
class ExplicitLayerNorm(nn.Module):
    def __init__(self, d_model, eps=1e-5):
        super().__init__()
        self.gamma = nn.Parameter(torch.ones(d_model))
        self.beta = nn.Parameter(torch.zeros(d_model))
        self.eps = eps

    def forward(self, x):
        mean = x.mean(dim=-1, keepdim=True)
        var = x.var(dim=-1, keepdim=True, unbiased=False)
        return self.gamma * (x - mean) / torch.sqrt(var + self.eps) +
         ↪  self.beta

layer_norm = nn.LayerNorm(d_model)
```

The original Transformer uses post-norm sublayers:

$$\text{LayerNorm}\big(x + \text{Sublayer}(x)\big).$$

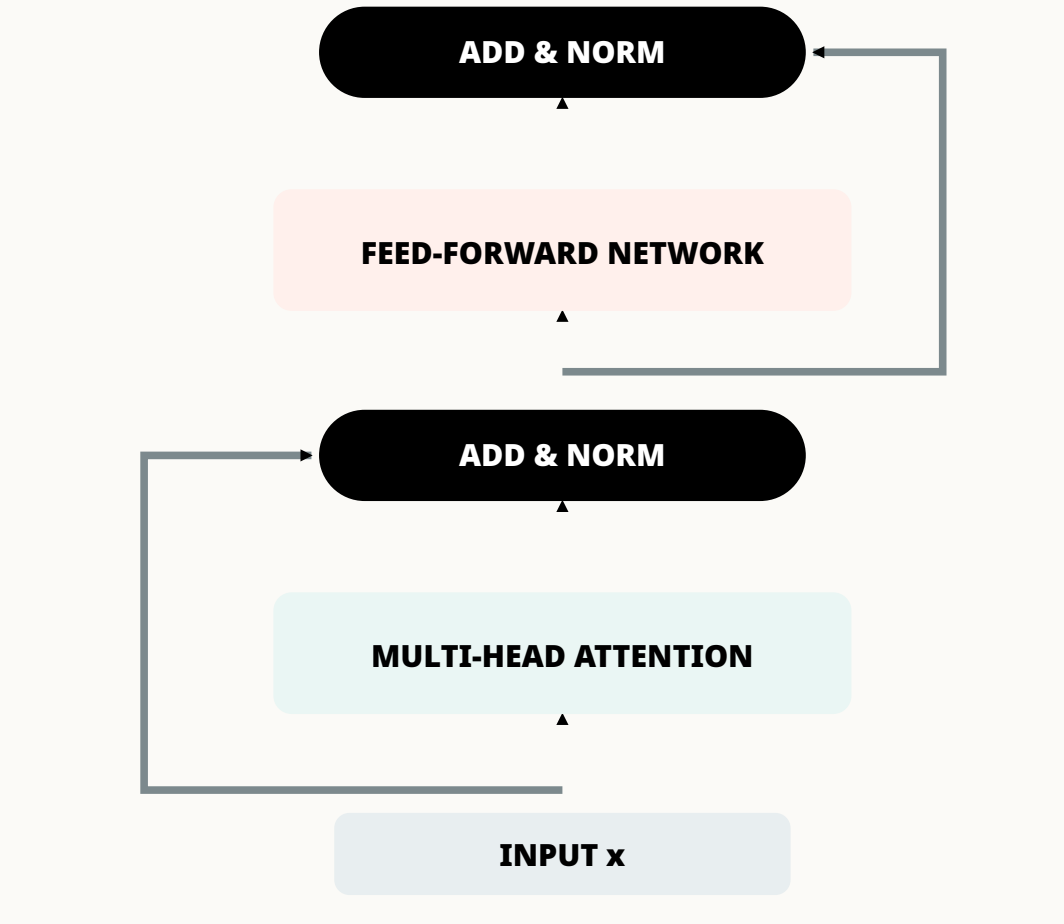


Figure 6: The classical post-norm block used in the lecture: each learned sublayer has a residual route followed by Add & Norm.

Many later large models use pre-norm. The distinction matters for optimization, but it does not change the conceptual roles of attention, FFN, residual routing, and normalization.

5 Encoder and decoder stacks

5.1 The decoder

A decoder block contains masked multi-head self-attention, Add & Norm, a position-wise FFN, and another Add & Norm. Repeating the block L times gives

$$Z^{(\ell)} = \mathrm{DecoderBlock}_{\ell}(Z^{(\ell-1)}).$$

The final states pass through a vocabulary projection and softmax to produce next-token probabilities. Inputs are shifted target tokens plus position representations.

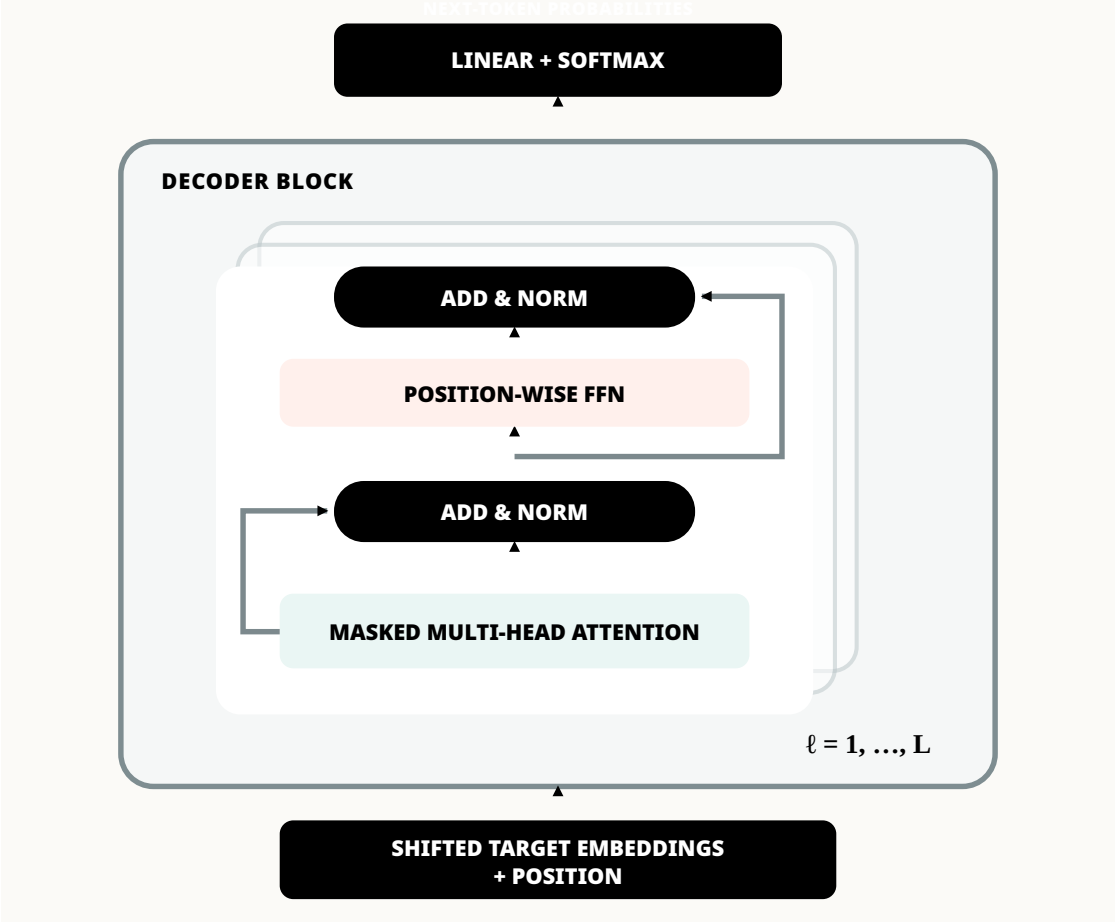


Figure 7: A decoder is a stack of repeated decoder blocks. Masked self-attention, residual normalization, and the FFN repeat before the final Linear + Softmax head.

5.2 Why the encoder has no causal mask

In translation, the complete source sentence is available before generation begins. The decoder must not inspect future French target words, because those are answers it must predict. The encoder, however, should use the complete English sentence: its job is to construct informed source representations.

For “She visited the bank to deposit cash,” the representation of *bank* benefits from both *visited* on its left and *deposit cash* on its right. Thus encoder self-attention uses the full $T \times T$ score matrix without a triangular causal mask.

An encoder block contains unmasked multi-head self-attention, Add & Norm, a position-wise FFN, and Add & Norm. Repeating it gives

$$X^{(\ell)} = \mathrm{EncoderBlock}_{\ell}(X^{(\ell-1)}).$$

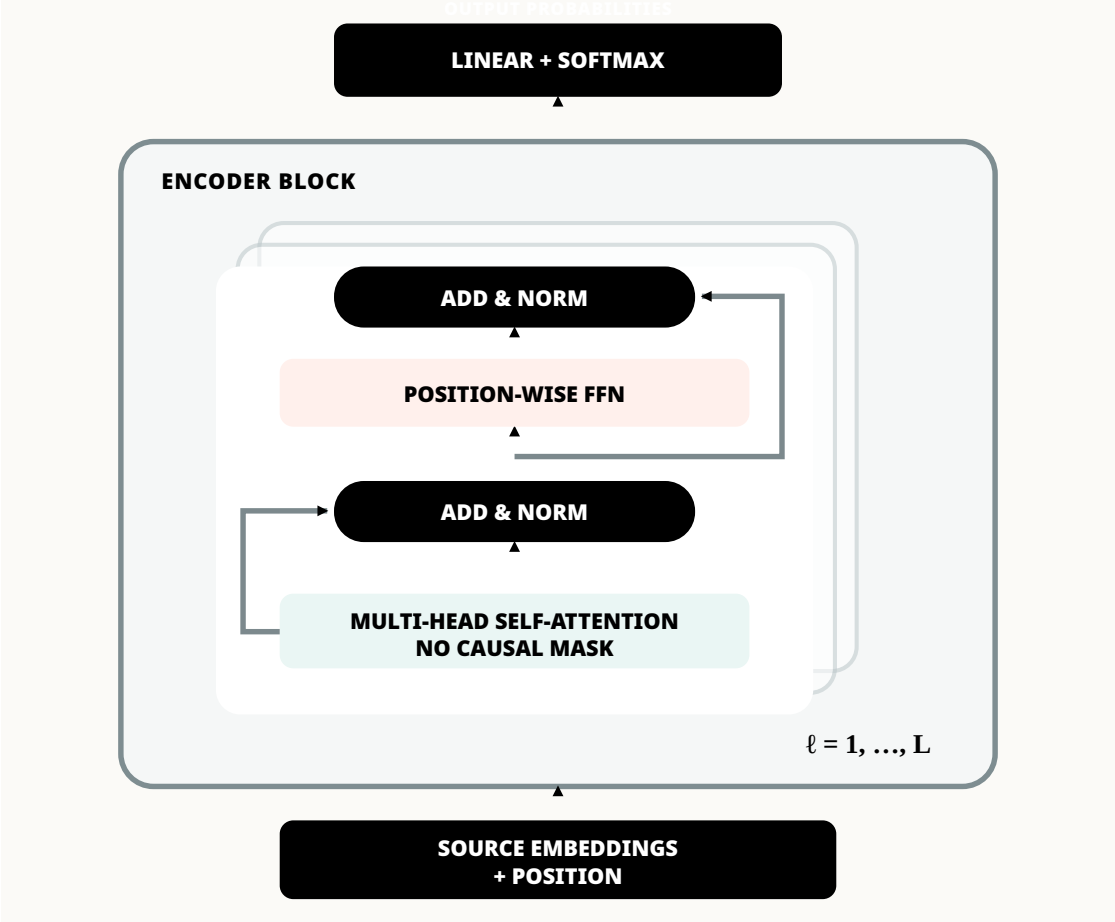


Figure 8: The corresponding encoder stack uses the same repeated-block visual language but removes the causal mask, allowing full source context.

6 Cross-attention connects both towers

Let H_{enc} be the complete encoder memory and Z_{dec} the current decoder representation. Cross-attention uses

$$Q = Z_{dec}W_Q, \qquad K = H_{enc}W_K, \qquad V = H_{enc}W_V,$$

$$\text{CrossAttn}(Z, H) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V. \tag{5}$$

Mechanism	Query source	Key/value source
Encoder self-attention	encoder	encoder
Decoder masked self-attention	decoder prefix	decoder prefix
Cross-attention	decoder	encoder memory

At each target position, the decoder query expresses what information is needed now. It scores every encoded source key, then retrieves a weighted mixture of source values. Each decoder block places this cross-attention sublayer between masked self-attention and the FFN.

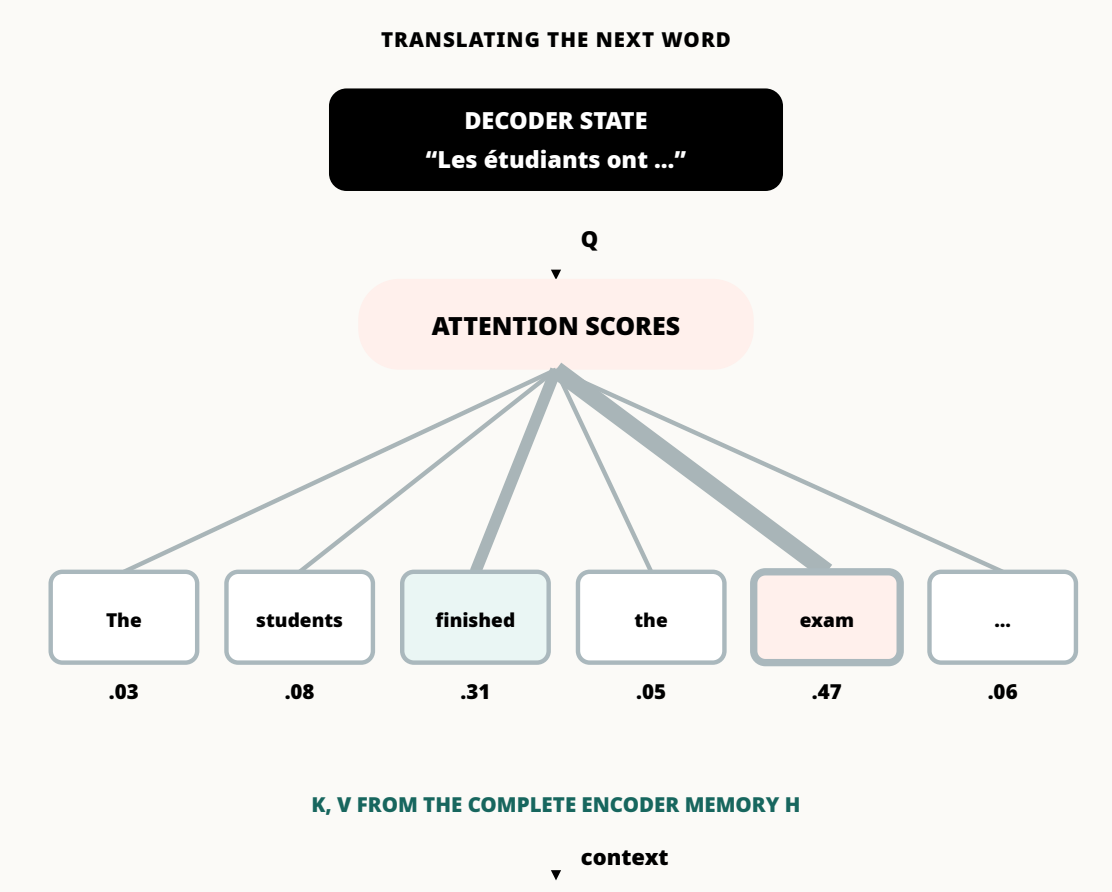


Figure 9: Cross-attention keeps the origins inspectable: the decoder supplies the query while the complete encoder memory supplies keys and values.

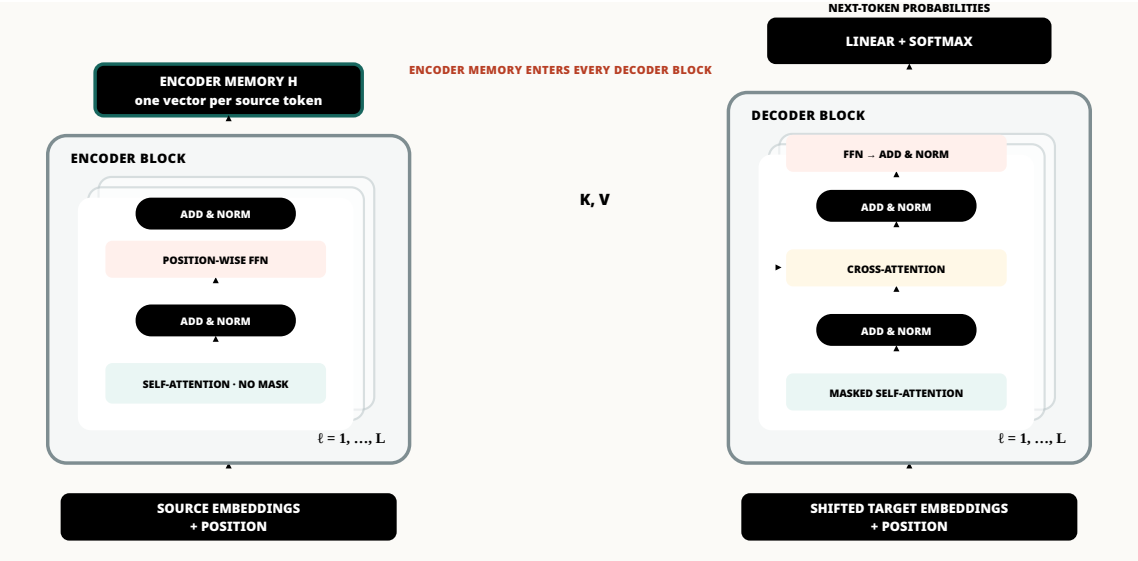


Figure 10: The complete encoder–decoder Transformer. The curved connection makes the decisive architectural fact visible: encoder memory enters the cross-attention sublayer of every decoder block.

For a complementary implementation-oriented derivation, see the Annotated Transformer and PyTorch's language translation tutorial.

7 Three Transformer families

Family	Visibility pattern	Typical output	Representative applications
Encoder-only	full bidirectional input	representation or label	classification, tagging, extractive QA, retrieval
Decoder-only	causal prefix	next-token distribution	generation, dialogue, code completion, in-context learning
Encoder–decoder	full source + causal target	target sequence	translation, summarization, generative QA

BERT exemplifies encoder-only pretraining, GPT-3 exemplifies decoder-only autoregressive modeling, and T5 develops a unified encoder–decoder text-to-text framework. The family is determined primarily by the permitted communication pattern and the form of the desired output.

8 Evidence and limitations

8.1 Translation results

The original Transformer achieved 28.4 BLEU on WMT 2014 English–German and 41.8 on English–French. The former improved over the previously reported best systems, including ensembles, by more than two BLEU points. The large model trained for 3.5 days on eight GPUs (Vaswani et al. 2017). These results mattered because the architecture combined quality with a much more parallel training computation than recurrence.

8.2 Transfer across tasks

BERT-Large reported a GLUE score of 80.5, SQuAD 1.1 F1 of 93.2, and SWAG accuracy of 86.3. The important result was not only each number: one pretrained bidirectional encoder could be fine-tuned across many language-understanding tasks. T5 extended this transfer principle by representing every task as text-to-text and reported gains across GLUE, SuperGLUE, question answering, summarization, and translation.

> Benchmark metrics are not interchangeable. Compare systems within the same dataset, split, and metric—not a BLEU score directly with an F1 or accuracy score.

8.3 Full attention is quadratic

With $Q, K \in \mathbb{R}^{T \times d_k}$,

$$S = QK^\top / \sqrt{d_k} \in \mathbb{R}^{T \times T}.$$

There are T^2 query–key scores per head and example, and the classical per-layer attention time is $\mathcal{O}(T^2 d)$ (Vaswani et al. 2017). Doubling context length quadruples the number of scores:

Context length T	Scores T^2
128	16,384
256	65,536
512	262,144
1,024	1,048,576

This limitation motivates sparse attention, linear attention, recurrence or memory, and optimized exact algorithms. Useful entry points include Sparse Transformers, Longformer, Transformers are RNNs, and FlashAttention. These approaches make different trade-offs: some approximate or restrict the attention pattern; FlashAttention computes exact attention while reducing memory traffic.

9 Synthesis

The complete architecture now follows from the limitations we repaired:

1. **Position** distinguishes order.
2. **Multi-head attention** communicates across tokens.
3. **The FFN** transforms each token nonlinearly.
4. **Causal masking** prevents target leakage.
5. **Residual connections and LayerNorm** make depth trainable.
6. **The encoder** reads a complete input.
7. **The decoder** generates from a prefix.
8. **Cross-attention** lets the decoder retrieve encoder memory.

The remaining question is not architectural but statistical: how can a large Transformer learn useful language representations from text without requiring a human label for every example? The next chapter answers this through **pretraining and BERT**.

9.1 Further reading

- Stanford CS224N · Attention and Transformers
- Vaswani et al. · Attention Is All You Need
- Jay Alammar · The Illustrated Transformer
- Harvard NLP · The Annotated Transformer
- PyTorch · Transformer building blocks
- Dive into Deep Learning · Transformers
- BERT paper
- T5 paper
- FlashAttention paper

References

Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. “Attention Is All You Need.” *Advances in Neural Information Processing Systems* 30. https://arxiv.org/abs/1706.03762.